VC5

資料、風險與成本

VC5 — Data, Risk & Cost

這一週回答三個你回到單位一定會被問的問題：
資料存到哪去了？能不能貼病歷？這樣一個月要多少錢？

Vibe Coding 專班 · 第 5 週 · 混合程度班 | 謝楠楨

這一週結束時，你手上會多出三樣東西

1 一個能存紀錄的成品

- 最近 10 筆試算紀錄，重新整理後還在
- 而且你說得出「這些資料存在哪裡」
- 無痕模式下會明確告訴使用者存不起來
- 這就是 v1.1 ，期末專題的基礎
- 範例 \v1.1 可以對照

2 金鑰的正確擺法

- .env ＋ .gitignore ＋平台環境變數
- 而且你會自己搜一次儲存庫確認沒外流
- 這是期末評分的紅線項目
- 搜得到金鑰，該項直接零分
- 本週要養成「部署完就搜一次」的習慣

3 一套可以回答別人的說法

- 什麼資料能貼進 AI 工具、什麼不能
- 去識別化之後還是不能貼的情況
- 一個月大約要花多少錢
- 回單位推動時，這三題一定會被問
- 答不出來，案子就推不動

第 3 項是這一週跟前四週最不一樣的地方。前面教的是「怎麼做出來」，這一週教的是「做出來之後，怎麼跟別人交代」——而後者往往才是決定案子能不能落地的關鍵。

本週地圖：哪幾頁要讀、哪幾頁要動手

頁	段落	怎麼用	預估時間
4–10	PART 01　資料要存在哪裡	讀。三個層級與各自的代價	20 分
11–15	PART 02　接外部資料與 CORS	讀。含「靜態網站藏不住金鑰」	15 分
16–21	PART 03　金鑰與環境變數	讀。本週的紅線題	15 分
22–28	PART 04　什麼資料能進 AI 工具	讀。回單位最會被問的一節	20 分
29–33	PART 05　成本與用量	讀。怎麼估、怎麼省	12 分
34–42	PART 06　動手做（Step 1–6）	打開你的 v1.0，跟著做	70–90 分
43	卡住了？照症狀查　／　常見的四個誤解	卡住時再翻	5 分
44–46	自我檢核、本週交付與回顧	交作業前一定要看	10 分

這一週的觀念頁比前幾週多（31 頁），動手頁較少。理由是：資料與風險的判斷沒辦法靠做一次就學會，它需要先建立一套判準——而那套判準會跟著你回到工作現場。

PART 01

資料要存在哪裡

Where Does the Data Live

「存起來」聽起來是一個功能，其實是一個治理決定。

資料一旦離開使用者的瀏覽器，你就要開始回答一整串問題。

資料存放的三個層級：越往上，要回答的問題越多

第③層　院內系統

資料留在醫院自己的機房

- 要走資訊室
- 時程以月計

第②層　雲端資料庫

Supabase 、 Firebase 等第三方服務

- 多人共用
- 要設權限

第①層　瀏覽器本機

localStorage ， 只存在使用者自己的電腦

- 不送出任何資料
- 本週作業停在這層

判斷用哪一層的一句話

「別人需不需要看到我存的東西？」不需要 → 第①層就夠了。多數個人用的小工具都停在這裡

往上一層的代價

第②層開始要回答：資料在哪個國家？誰看得到？出事誰負責？備份多久？——這些是治理問題，不是技術問題

為什麼作業停在第①層

第①層不會把任何資料送出去。這讓你可以專心練技術，不必先處理治理問題

期末專題呢

也建議停在第①層。真的需要多人共用時，請在報告裡寫清楚你打算怎麼處理那一串問題

這三層不是「越高越好」，而是「越高責任越重」。實務上最常見的錯誤是：一開始就選了第②層（因為聽起來比較專業），然後卡在資安審查上動彈不得。

第①層　瀏覽器本機（localStorage）：本週要做的

項目	說明
資料存在哪	使用者自己的瀏覽器裡。完全不會送到任何伺服器
最大的優點	沒有任何隱私問題要處理——因為資料從來沒有離開過那台電腦。 這也是為什麼本課程的作業停在這一層
換裝置看得到嗎	看不到。換一台電腦、換一個瀏覽器、清除瀏覽資料，紀錄就沒了
容量限制	各家瀏覽器不同，大約 5–10 MB 。存純文字的話很夠用
什麼時候會失效	無痕模式、瀏覽器隱私設定、以及用 file:// 直接開檔時， 可能會直接丟出錯誤——所以程式一定要包 try/catch
使用者的誤解	使用者常以為「存起來了」＝「雲端保存」。 介面上最好明說「紀錄只留在這台裝置」

> 最後兩列是本週 Step 1 要實作的重點：不只是「能存」，還要在「存不起來的時候明確告訴使用者」。沉默失效比功能壞掉更糟——這一點 VC2 已經講過一次了。

第②層　雲端資料庫：一旦選了它，你要回答的五個問題

#	問題	為什麼這一題不能跳過
1	資料實際存在哪個國家?	跨境傳輸涉及個資法規。多數雲端服務預設在美國或新加坡
2	誰看得到？權限怎麼設?	預設常常是「知道網址就看得到」。不設權限等於公開
3	出事的時候誰負責?	資料外洩時，是你、是單位、還是服務商？合約寫了什麼?
4	備份多久？刪得掉嗎?	使用者要求刪除時，你刪得乾淨嗎？備份裡的那一份呢?
5	服務停掉怎麼辦?	免費方案可能隨時調整或終止。資料匯得出來嗎?

這五題沒有一題是技術問題。它們是治理問題——而治理問題不是「做得出來」就能解決的，需要單位層級的決定。這也是為什麼本課程建議停在第①層：不是做不到，是不該由一門六週的課草率帶過。

第③層　院內系統：什麼時候才會走到這裡

項目	說明
什麼情況需要	要處理真實個案資料、要跟 HIS 或長照系統介接、 或單位要求資料不得離開院內
時程	以月計，不是以週計。需求訪談、資安審查、採購或委外、 測試、上線——每一關都有人要簽字
你的角色會變成什麼	從「做出來的人」變成「說得清楚需求的人」。 而這恰好是這門課練到的能力（VC1 的規格）
這門課能幫上什麼	用前四週學的東西做一個原型，拿著它去談需求—— 比拿一份文字規格有效十倍。這是 Vibe Coding 最實際的用途之一
不能做什麼	不能把課堂做的原型直接拿去用。它沒有經過資安審查、 沒有稽核軌跡、沒有權限控管

> 第三列與第四列值得記住：這門課最務實的產出，往往不是那個小工具本身，而是「你可以拿著一個看得見、摸得到的原型去跟資訊室談」。原型讓抽象的需求變成具體的討論。

你的專案該用哪一層?

你想加上「把資料存起來」這個功能

1. 別人需不需要看到你存的東西?
   - 是 → 不需要（只有使用者自己看） → 第①層， localStorage 就夠了
     例：試算紀錄、個人偏好設定、草稿
     本課程的作業與期末專題都建議停在這裡
   - 否 ↓
2. 需要多人共用，但資料是公開或模擬的?
   - 是 → → 第②層可以考慮，但先把那五個問題想過一遍
     課堂專案請用模擬資料，不要用真實個案
     並在報告裡寫清楚：資料存在哪、誰看得到
   - 否 ↓
3. 涉及真實個案資料?
   - 是 → 停。不論第②層或第③層，都不是一門六週的課能處理的
     本課程紅線：一律不得使用真實個案資料
     真的需要時 → 走第③層，從資訊室開始談
   - 否 ↓
4. 還不確定
   - → 先做第①層。它不會把資料送出去，之後要換也不難
     通則：選最單純、能滿足現在需求的那一層
     複雜度要等到真的需要時再引入（ VC4 也講過同一個原則）

第一題就決定了八成的情況。多數個人用的小工具——包括本課程的試算器——真的只需要第①層。選了更高層而沒有實際需要，你買到的不是功能，是一整串要回答的問題。

想知道資料真的存在哪？用開發者工具看一眼

F12 → Application 分頁（Chrome / Edge）

```
Application
  ├─ Local Storage
  │    └─ https:// 你的網址
  │
  │        Key                  Value
  │        ltc-calc-history     [{"level":5,…}]
  │
  ├─ Session Storage
  ├─ Cookies
  └─ …


//  你可以在這裡：
//
//    ・看到實際存了什麼（就是一段文字）
//    ・直接刪掉某一筆，測試「沒有紀錄時」的畫面
//    ・確認換一個網域就看不到（各網站各自獨立）
```

1 它就是一張「文字對文字」的表

左邊一個名字、右邊一段文字，就這樣
沒有想像中複雜

2 綁在「網域」上

a 網站存的， b 網站看不到
換一個網域就是全新的一份

3 可以直接刪來測試

刪掉之後重新整理，就能測「沒有紀錄」的畫面
比等它自然清空快得多

4 這裡看得到＝沒有送出去

資料在這一格裡，代表它從來沒離開這台電腦
這就是第①層最大的價值

這一頁是 VC2 開發者工具的延伸應用：「資料存在哪裡」不是一個抽象的說法，你可以實際打開來看。回單位被問到時，這一招比任何解釋都有說服力——直接開給對方看。

PART 02

接外部資料與 CORS

APIs & CORS

想接外部資料（政府開放資料、天氣、匯率）時，會撞到一堵牆。
這一節解釋那堵牆是什麼、為什麼存在、以及什麼時候繞不過去。

一次「跟外部要資料」發生了什麼事

1

① 你的網頁

在瀏覽器

2

② 發出請求

「請給我

3

③ 對方伺服器

檢查、

4

④ 回傳

資料或

5

⑤ 你的網頁

拿到資料

這五步在正常情況下是幾百毫秒的事。但第 ③ 步藏了一個檢查：對方會看「是誰在要這筆資料」——如果它不認識你的網域，瀏覽器就會把回應擋掉。那就是 CORS 。

CORS：為什麼你會看到這段紅字

Console 裡實際會看到的錯誤

```
Access to fetch at 'https://api.example.com/data'
from origin 'https:// 你的網址 .github.io'
has been blocked by CORS policy:

No 'Access-Control-Allow-Origin' header is
present on the requested resource.


//  翻成中文：
//
//    你的網頁想跟 api.example.com 要資料，
//    但對方沒有在回應裡說「我允許這個網域來拿」，
//    所以瀏覽器把回應攔下來了。
//
//    注意：資料其實已經送回來了，
//    是「瀏覽器」不給你用，不是對方沒給。
```

1 這不是你的程式寫錯

語法完全正確，程式也真的送出了請求
問題出在「對方允不允許」

2 誰擋的

是瀏覽器擋的，不是伺服器
這是瀏覽器保護使用者的安全機制

3 為什麼要有這個機制

防止惡意網站偷偷用你的身分去別的網站拿資料
沒有它的話，任何網頁都能讀你登入中的資料

4 怎麼判斷是不是 CORS

錯誤訊息裡有 CORS 或 Access-Control 這兩個字
而且 Network 分頁看得到請求真的送出去了

這一頁的重點是「不要花時間改自己的程式」。CORS 錯誤幾乎不可能靠改前端解決——它是對方的設定。看到這段紅字，就直接跳到下一頁看三種處理方式。

撞到 CORS 的三種處理方式

做法	怎麼做	適用與代價
① 換一個 允許跨域的來源	很多政府開放資料平台有提供允許跨域的端點。 查一下對方的說明文件有沒有 CORS 相關設定	最省事。先試這個。 課堂專案多半這樣就解決了
② 先下載成 靜態檔案	把資料下載成 .json 或 .csv 放進你的專案， 程式直接讀本機的檔案	適合「資料不常變」的情況（例如給付額度表）。 代價：資料更新要手動換檔
③ 加一層 自己的後端	你的網頁 → 你的後端 → 對方 API。 後端之間沒有 CORS 限制	唯一的通解，但你的專案就從「靜態」 變成「需要後端」（見 VC4 第 24 頁）
不要做的事	網路上會查到「用某個代理服務繞過去」的做法	不要用在任何跟醫療有關的東西上—— 你的資料會經過一個不認識的第三方

本課程的建議：課堂專案用 ① 或 ②。需要 ③ 的話，代表你的專案已經超出「靜態網站」的範圍，成本與責任都會跳一級——請先確認真的有這個必要。

靜態網站藏不住任何金鑰

	× 前端（送到瀏覽器的東西）				○ 後端（留在伺服器上）			
你以為藏起來了	金鑰寫在 calculator.js	→ 只有程式看得到吧	→ 按 F12 → Sources	→ 整個檔案都在那裡	金鑰只放後端／函式	→ 前端呼叫自己的後端	→ 後端才去呼叫外部 API	→ 瀏覽器看不到金鑰
換個藏法	用變數名偽裝一下	→ 或用簡單的編碼	→ 按 F12 → Network	→ 請求裡原形畢露	同上	→ 金鑰從來沒有送到瀏覽器	→ 看不到就是看不到	→ 唯一安全的做法
用環境變數	寫進 .env	→ 但建置時被打包進去	→ 一樣看得到	→ 環境變數不等於安全	環境變數設在後端執行環境	→ 不是前端的建置環境	→ 兩者完全不同	→ 這是關鍵區別

第三列是最常見的誤解：把金鑰放進 .env 只解決了「不要推上 GitHub」，沒有解決「不要送到瀏覽器」。前端專案的建置過程會把環境變數的值直接寫進打包後的檔案裡——使用者按 F12 照樣看得到。

PART 03

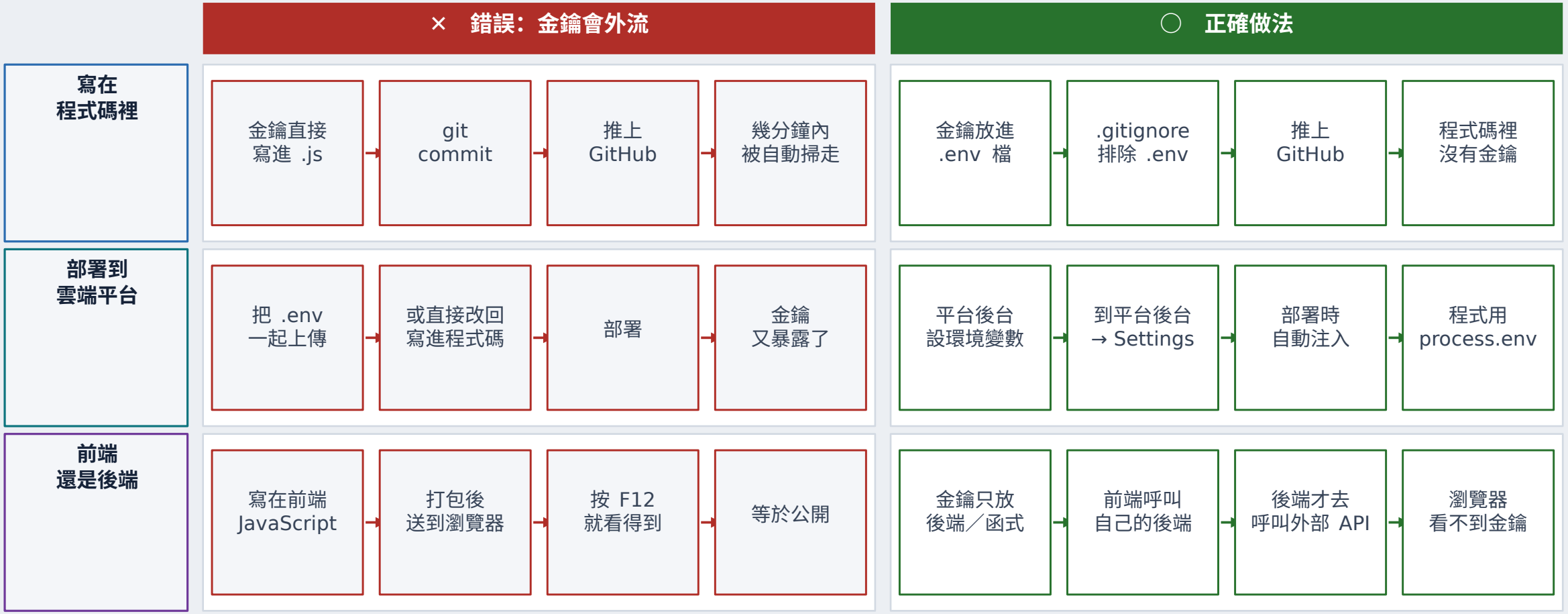
金鑰與環境變數

Secrets & Env Vars

這是最常見、也最嚴重的資安錯誤，而且完全可以避免。

本週的驗收條件之一：助教會實際搜一次你的儲存庫。

金鑰不能寫在程式碼裡：三種情境的錯與對



萬一已經推上去了怎麼辦：第一件事是「到服務商後台把那把金鑰撤銷並重新產生」，不是去刪 commit。刪掉 commit 也救不回來——GitHub 的歷史紀錄與各種鏡像早就存下來了。撤銷金鑰才是唯一有效的處置。

.env 是什麼：三個檔案的分工

專案資料夾裡的三個檔案

```
 .env                真正的金鑰放這裡

    OPENAI_API_KEY=sk-abc123 真正的金鑰
    DB_PASSWORD= 真正的密碼


 .env.example        給別人看的樣板

    OPENAI_API_KEY= 你的金鑰放這裡
    DB_PASSWORD= 你的密碼放這裡


 .gitignore          排除清單

    .env
    .env.local
    node_modules/
```

1 .env　絕對不推上去

裡面是真的金鑰
被 .gitignore 排除掉

2 .env.example　要推上去

它告訴接手的人「你需要哪幾把金鑰」
但裡面沒有真的值

3 .gitignore　要推上去

它是專案設定的一部分
少了它，別人 clone 下來也會外流

4 順序很重要

.gitignore 要在「第一次 commit 之前」設好
已 commit 過的檔案，之後排除也還在歷史裡

.env.example 這個習慣值得學起來：它讓接手的人知道「這個專案需要哪些金鑰」，卻不會洩漏任何一把。這也是 VC6 交接文件的一部分——沒有它，別人 clone 下來會完全不知道要設什麼。

部署平台的環境變數設在哪裡

平台	路徑	注意
Vercel	專案 → Settings → Environment Variables	設完之後要重新部署一次才會生效
Netlify	Site configuration → Environment variables	同上。可以分 production 與 preview 兩組
Firebase	用指令設定，或在 Functions 的設定檔裡	要用 CLI，多一個步驟
GitHub Pages	沒有這個功能	它只服務靜態檔案，沒有執行環境—— 所以也就沒有「安全存放金鑰」的地方
本課程的專案	完全不需要任何金鑰	所以這一節是為了「以後」而學。 但 Step 4 的「搜一次儲存庫」現在就要做

倒數第二列點出一個重要的界線：GitHub Pages 之類的純靜態託管，本質上就沒有地方能安全放金鑰。需要金鑰＝需要後端——這兩件事是綁在一起的，不是設定問題。

已經把金鑰推上去了，怎麼辦

你在儲存庫裡搜到了自己的金鑰

1. 第一件事：立刻到服務商後台撤銷那把金鑰
   - 做法：不要先去刪 commit 。撤銷才是唯一有效的處置
     撤銷後重新產生一把新的，舊的那把立即失效
     幾乎所有服務商的後台都有「 Revoke 」或「刪除並重新產生」
2. 第二件事：確認影響範圍
   - 做法：這把金鑰能做什麼？被用了會有什麼後果（花錢？讀資料？）
     到服務商後台看用量紀錄，確認有沒有異常
     如果涉及資料存取，依單位規定通報
3. 第三件事：修好程式碼
   - 做法：把金鑰移到 .env ，設好 .gitignore ，重新 commit
     到部署平台後台設環境變數，用新的那把金鑰
     重新部署，確認功能正常
4. 要不要清理 Git 歷史
   - 做法：可以做，但不要當成主要手段——金鑰已經撤銷了，歷史裡那串字沒有用
     清理歷史會改寫所有 commit 編號，多人協作時會造成混亂
     課堂專案的建議：撤銷金鑰、修好程式碼，歷史留著就好

順序不能顛倒。很多人的直覺是「趕快把那個 commit 刪掉」——但刪 commit 需要幾分鐘，而自動掃描機器人找到公開儲存庫裡的金鑰通常只要幾十秒。撤銷金鑰是唯一跑得贏它的動作。

四類風險與對應做法（回單位推動時會被問到）

風險	具體情況	對應做法
資安	AI 可能產生含漏洞的程式碼； 可能寫入不安全的預設值	用 VC3 的審查檢核表逐項檢查； 金鑰一律走環境變數；部署後搜一次儲存庫
正確性	算錯不會報錯，會用正確的格式 顯示錯的數字	驗收條件 ＋ 測試 ＋ 覆蓋網格（VC3 整週）。 涉及專業判斷的部分一定要有人審過
隱私	貼進 AI 工具的內容會送到服務商； 雲端資料庫的權限預設可能是公開	不貼真實個案資料；資料停在第①層； 介面明說「紀錄只留在這台裝置」
維護	額度表、法規、 API 都會變， 而程式碼不會自己提醒你	在 README 的「已知的限制」寫明哪些數字會過期。 這是 VC6 的內容

四類裡「正確性」與「維護」最容易被忽略，因為它們不會當場出事。但在醫療場域，一個安靜算錯的工具，比一個當掉的工具危險得多——後者至少你知道它壞了。

PART 04

什麼資料能進 AI 工具

What You Can Paste

回到單位之後，這是第一個會被問的問題。
答不出來，案子就推不動——而且問的人通常是對的。

個人版 vs 企業版：差在哪

比較項目	個人版 / 免費版	企業版 / 團隊版
資料會不會拿去訓練	多數個人版預設「可能會」， 通常有開關可以關掉	企業版通常契約保證不用於訓練
資料保留多久	多半保留一段時間（用於安全稽核）， 各家不同	可協商，部分方案支援零保留
誰看得到	服務商的系統；必要時人工審查	同左，但有合約與稽核條款約束
資料在哪個國家	多半在境外，且你通常不能選	部分方案可指定區域
有沒有合約可以看	通常只有線上條款，隨時可改	有正式合約與資料處理協議
這門課的立場	課堂與作業一律不貼真實個案資料	單位要導入的話，這是最低門檻
怎麼確認	到工具的隱私設定頁面實際看一次—— 不要憑印象，各家差異很大且常改	同左，另請單位法務或資訊室看合約

最後一列最重要：這些政策變動頻繁，而且各家差很多。本講義刻意不寫死任何一家的政策——請在使用前實際去看一次那個工具的隱私設定與條款，這個習慣比記住任何結論都有用。

什麼算「去識別化」

移除的東西	算不算去識別化	為什麼
姓名、身分證號、病歷號	這是最低限度，不是充分條件	移除直接識別資訊只是第一步
生日 → 只留年齡	有幫助	但罕見年齡（例如 103 歲）仍可能被指認
完整地址 → 只留縣市	有幫助	偏鄉地區 + 罕病 + 特定年齡，組合起來仍可能唯一
罕見疾病、特殊處置	移不掉——它本身就是關鍵資訊	「某院 2026 年唯一一例某罕病」= 已經識別了
時間資訊（住院日期等）	常被忽略	日期組合起來的識別力很高，尤其小樣本

一個實用的判準：「把這份資料交給一個熟悉本院業務的同事，他猜得出來是誰嗎？」猜得出來，就還沒去識別化。而在小型單位或罕見疾病的情境下，這一題的答案常常是「猜得出來」。

去識別化之後，仍然不該貼的三種情況

① 資料量小、樣本特殊

- 十筆以內的個案、或某個罕見組合——去識別化幾乎沒有效果
- 熟悉業務的人一看就知道是誰
- → 改用完全虛構的模擬資料，或只描述規則不貼資料

② 單位規定或合約沒有允許

- 「技術上做得到」不等於「制度上可以做」
- 多數醫療機構對「資料離開院內系統」有明確規範，即使去識別化也一樣
- → 先問資訊室或個資窗口，不要自己判斷

③ 你只是想「讓 AI 幫忙看一下」

- 這是最常見、也最容易出事的情境——因為它感覺很無害
- 貼一份去識別化的名單問「幫我整理成表格」，資料就已經送出去了
- → 需要的話，用結構相同但內容虛構的假資料去問做法，再自己套用

第 ③ 點的替代做法值得記住，而且很實用：你要的通常是「做法」，不是「AI 幫你處理這批資料」。用假資料問到做法，再自己在院內套用——結果一樣，但資料從來沒有離開過。

資料分級決策樹：這份資料能不能貼？

你手上有一份資料，想貼進 AI 工具

1. 裡面有真實個案資料嗎？（即使去識別化）
   - 是 → 有 → 停。本課程一律不貼
     實務上：先問單位的個資窗口或資訊室，不要自己判斷
     替代做法：用結構相同、內容虛構的假資料去問做法
   - 否 ↓

2. 是單位內部文件嗎？（未公開的流程、名冊、報表）
   - 是 → 依單位規定。多數機構對此有明確規範
     不確定就當作「不可以」——這一題的預設值應該是保守的
     可以的話：把敏感欄位換成假值再貼
   - 否 ↓

3. 是公開資料嗎？（法規、公告、公開統計）
   - 是 → 可以貼。長照給付基準、健保公告、政府開放資料都屬於這一類
     本課程的專案用的就是這一類
     但仍要自己查證數字，不要用 AI 給的（VC1、VC3 都講過）
   - 否 ↓

4. 是你自己寫的程式碼或文件？
   - 是 → 可以，但先確認裡面沒有金鑰、密碼、內部網址
     這是本週 Step 4 要做的：貼之前先自己搜一次
     含真實資料的檔案不要貼

這張決策樹請印出來。回到單位之後，它就是你跟同事解釋「為什麼這個可以那個不行」的依據。注意第二題的預設值是保守的——不確定時當作不可以，比事後解釋容易得多。

院內可控部署：什麼時候值得考慮

選項	是什麼	什麼時候值得
院內部署的小型模型	在醫院自己的機器上跑一個較小的模型，資料完全不出院	要處理真實資料、且量夠大時。對應 M0 講的「SLM 小型專用模型」趨勢
企業版 + 資料處理協議	用商業服務，但走正式合約，約定不訓練、可指定區域、可稽核	最常見的折衷。多數單位會走這條
混合	敏感的部分院內處理，一般的部分用外部服務	實務上最務實。例如：去識別化在院內做，完成後的統計分析才用外部工具
這門課的位置	完全不碰真實資料，所以三個選項都不需要	但你要知道它們存在——回單位討論時，「有哪些選項」比「我覺得應該可以」有用得多

這一頁的目的不是要你去做院內部署，而是讓你在會議上聽得懂、講得出來。當有人說「資料不能出去所以不能用 AI」時，你知道那不是二選一——中間有好幾種做法。

回單位之後，怎麼跟主管／資訊室解釋

對方會問	不好的回答	好的回答
「資料會不會外流？」	「不會啦，很安全」	「這個版本的資料完全存在使用者自己的瀏覽器裡，沒有任何對外連線。我可以現在打開開發者工具給你看」
「這是 AI 寫的，能信嗎？」	「AI 現在很強」	「我有一份驗證報告：測了哪 29 項、預期值從衛福部給付基準查來的、還有五條我確定沒驗到的限制」
「一個月要花多少？」	「應該不會很多」	「目前五週實際用了 X，估一個月約 Y，我已經設了用量上限 Z。真的要推廣的話成本結構是這樣…」
「以後誰維護？」	「我可以繼續看」	「README 裡寫了怎麼跑起來、資料從哪來、以及六條已知的限制。另一位同事照著跑起來過」
「那可以正式上線嗎？」	「應該可以吧」	「不行。這是原型，沒有經過資安審查、沒有稽核軌跡、沒有權限控管。但它可以用來談需求」

五個「好的回答」有一個共同結構：具體 + 有證據 + 說得出限制。而這三樣東西，正好就是你這六週一路做出來的（驗證報告、成本估算、README、已知的限制）。最後一列尤其重要——主動說出「不能上線」，比被問倒有說服力得多。

PART 05

成本與用量

Cost & Usage

「這樣一個月要多少錢？」——回單位一定會被問的第三個問題。
多數工具已從吃到飽轉向用量計費，估得出來很重要。

什麼操作最耗用量

操作	耗用程度	為什麼
代理模式的長對話	最貴	它會反覆讀檔、試錯、再讀檔。 一次「幫我重構整個專案」可能等於幾十次普通對話
把整個專案貼給它	貴	每一次對話都要把整份內容再讀一遍—— 對話越長，重複讀的量越大
產生長篇程式碼	中等	輸出的長度也算量，但通常比輸入少
單純問一個問題	最便宜	「這一行是什麼意思」這種問題幾乎不花錢
反覆重試同一件事	隱形的大宗	描述不清楚 → 它做錯 → 你再說一次 → 它再做。 這也是為什麼 VC1 花九頁講規格

最後一列值得注意：省錢最有效的方法不是「用便宜的模型」，而是「一次講清楚」。寫好規格、給足證據——這兩件事同時省下你的時間與用量，而且省得比任何技巧都多。

怎麼估一個專案大約要多少

步驟	怎麼做	本課程專案的實際情況
① 先跑通 再看用量	用免費額度把整個流程做過一遍， 然後到工具後台看實際用了多少	六週的課堂專案，多數學員在免費額度內就做完了
② 抓一個 單位成本	「做完一個像這樣的小工具」大約用掉多少？ 把它當成一個基準	以本課程的試算器為例： 從零到部署，約等於幾十次來回對話
③ 乘上 預期次數	一個月會做幾個這樣的東西？ 或這個工具會被叫用幾次？	課堂專案：一學期一個 → 幾乎不用估
④ 加上 意外的部分	抓 1.5–2 倍。實際上總是比估的多—— 因為卡住重試的次數估不準	這一條在任何專案估算都適用，不限 AI
⑤ 設一個 上限	多數工具可以設用量上限或告警。 開課第一週就設好	強烈建議：避免一次失控的長對話燒掉整個月的額度

第 ⑤ 步請現在就做。代理模式一次跑錯方向，可能在十分鐘內用掉你原本以為夠用一個月的額度——設上限是唯一能真正防住的手段。

省用量的三個習慣（同時也省你的時間）

① 一次講清楚，而不是慢慢問出來

- VC1 的五行規格 + 驗收條件，就是在做這件事
- 描述不清楚 → 它做錯 → 你再說一次 → 它再做：這一輪的成本是原本的三倍
- 「先說明你打算怎麼做」這一句，也是在用最少的量攔下誤解

② 對話太長就開新的

- 每一次回應都要把整段對話再讀一遍——對話越長，每一次都越貴
- 換一個主題時就開新對話，把需要的檔案重貼一次
- 附帶好處：它比較不會忘記前面（M7 講的上下文長度限制）

③ 分清楚「要它做」與「問它」

- 「這一行是什麼意思」——幾乎不花錢，可以盡量問
- 「幫我重構整個專案」——最貴的一種，做之前先確認真的需要
- VC4 的習慣在這裡也有用：先 commit，改壞了 revert，不必重跑一次

三個習慣的共同點：它們本來就是好的工作方式，省用量只是附帶的效果。不要為了省錢而把需求講得含糊——那會讓你多花好幾倍的量在來回釐清上。

何時該用／不該用：醫療場域的判準

適合	不適合
快速原型與概念驗證	會影響真實臨床決策的計算邏輯
重複性高、邏輯明確的資料處理腳本	處理真實個案資料的系統
你不熟的技術棧，想先做出東西再學	你完全無法驗證正確性的專業計算
一次性的分析工具、視覺化小程式	需要通過法規查核或稽核的產出
既有程式碼的解釋與文件補寫	效能敏感、需要精細最佳化的程式
低風險、做壞了重來成本很低的任務	長期維護、多人協作的核心系統

> 判斷的通則一句話：如果這段程式碼算錯了，會有人受影響嗎？會的話，就必須有人能完整讀懂並負責——不能停在「AI 寫的、跑起來沒問題」。這句話 VC1 講過一次，六週後你應該已經能自己說出來了。

PART 06

動手做： Step 1–6

Hands-on

請打開你上週部署好的 v1.0 專案。

Step 4 「自己搜一次儲存庫」是本週的驗收條件——助教會實際搜。

Step 1　加上「存下試算紀錄」

你現在要做什麼　▶　請 AI 加上最近 10 筆的試算紀錄，存在瀏覽器本機

可以直接照抄

請在試算器加上「最近的試算紀錄」功能：

1. 每次按計算成功後，把「等級、身分別、自負額、時間」存進 `localStorage`，最多保留 10 筆，最新的在最上面。
2. 一開啟頁面就把紀錄畫出來；沒有紀錄時不要顯示這一區。
3. 提供一個「清除」按鈕。
4. 所有 `localStorage` 存取都要用 `try/catch` 包起來。
5. 如果這個瀏覽器不能用 `localStorage`（無痕模式等），請在畫面上明確告訴使用者「紀錄不會被保存」。

請只修改 `index.html`，不要更動 `calculator.js`。

你應該看到什麼

畫面下方多出一個「最近的試算紀錄」區塊。
算三次之後會列出三筆，最新的在最上面。
重新整理頁面，紀錄還在。
按「清除」之後區塊消失。
test.html 重跑，29 項應該全綠——
因為 calculator.js 一行都沒改。

沒看到的話，先檢查這幾件事

① 重新整理之後紀錄不見了
→ 可能是無痕模式，或用 file:// 開檔。
用部署後的網址測比較準
② 沒有出現「不能保存」的提示
→ 要求它補上第 5 點
③ 測試變紅了 → 它動到 calculator.js 了，
要它還原

第 4、5 點是這一步的重點，而且是刻意要求的。「存不起來卻不說」比「不能存」更糟——使用者以為紀錄留著了，其實沒有。這一點 VC2 講過一次（沉默失敗），這裡是它在資料場景的版本。

Step 1-2　驗證紀錄功能：四個情境

你現在要做什麼　▶　不要只試「正常存得起來」那一種

四個情境

① 正常：算三次 → 出現三筆，最新的在最上面
② 上限：連算 14 次 → 應該只留 10 筆
③ 重新整理：紀錄還在
④ 無痕模式：開一個無痕視窗試 → 應出現「不會被保存」的提示

怎麼開無痕視窗

Windows：Ctrl + Shift + N（Chrome）／Ctrl + Shift + P（Firefox）
macOS：Command + Shift + N
或用瀏覽器右上角選單找「新增無痕式視窗」

你應該看到什麼

四個情境都符合預期。
特別是第 ④ 個——
無痕模式下試算功能仍然正常，
只是多出一行提示說紀錄不會保存。
這才叫「優雅地失效」。

沒看到的話，先檢查這幾件事

① 無痕模式下整頁掛掉
→ try/catch 沒包好。
要它把所有 localStorage 存取都包起來
② 無痕模式下紀錄居然存得起來
→ 正常的。各家瀏覽器行為不同，
重點是「不能存的時候要說」

第 ② 個情境（上限 10 筆）很容易被忘記。沒有上限的話，紀錄會一直累積直到 localStorage 滿了——然後在某一天突然開始丟出錯誤。這是典型的「現在不會出事、以後才出事」的問題。

Step 2　把一把金鑰放進 .env（練習用）

你現在要做什麼　▶　本課程專案不需要金鑰，這一步是為了練習正確的擺法

① 在專案資料夾建立 .env

檔名就叫 .env（前面有一個點，沒有副檔名）

內容（隨便寫一個假的來練習）：

```
DEMO_API_KEY=sk-this-is-a-fake-key-for-practice
```

② 確認 .gitignore 有排除它

打開 .gitignore，確認裡面有這兩行：

```
.env
.env.local
```

③ 建立 .env.example 給別人看

```
DEMO_API_KEY= 你的金鑰放這裡
```

你應該看到什麼

git status 之後，
.env 不應該出現在待提交的清單裡
（.env.example 應該要出現）。

這就是正確的狀態：
真的金鑰留在本機，
樣板檔推上去給別人看。

沒看到的話，先檢查這幾件事

① .env 還是出現在 git status 裡
→ 檢查 .gitignore 有沒有存檔；
或者它之前已經被 commit 過了
② Windows 上建不出「.env」這個檔名
→ 在編輯器裡「另存新檔」打完整檔名；
或 cmd： type nul > .env
PowerShell： New-Item .env

順序很重要：.gitignore 要在「第一次 commit 之前」就設好。如果 .env 曾經被 commit 過，即使現在加進 .gitignore，它還是留在歷史裡——這是最常見的金鑰外流原因。

Step 3　在部署平台後台設環境變數

你現在要做什麼　▶　練習用。本課程專案不會真的讀它，但流程要走一次

Vercel

專案 → Settings → Environment Variables
Name： DEMO_API_KEY　Value： （隨便一個假值）
存好之後要重新部署一次才會生效

Netlify

Site configuration → Environment variables → Add a variable

GitHub Pages 使用者

它沒有這個功能——純靜態託管沒有執行環境。
這一步請改成「讀一次第 19 頁，理解為什麼沒有」。
需要金鑰＝需要後端，這兩件事是綁在一起的。

你應該看到什麼

平台後台的環境變數清單裡，
多出你剛才設的那一筆。

重點不是這個值有沒有被用到，
而是你知道「以後真的需要金鑰時，
它應該放在這裡，不是放在程式碼裡」。

沒看到的話，先檢查這幾件事

① 找不到那個設定頁
→ 平台介面常改版。搜尋 Environment 或 Variables 這兩個字
② 設完之後專案壞了
→ 不會。沒有程式碼讀它的話，設了也不會有任何影響

這一步刻意用假值。目的是讓你走過一次流程，而不是真的在課堂上使用任何金鑰——真正需要金鑰的專案，安全要求會比一門六週的課高得多。

Step 4　自己搜一次儲存庫（本週驗收條件）

你現在要做什麼　▶　到 GitHub 儲存庫頁面，搜尋四個關鍵字

兩種搜法

① GitHub 網頁：在儲存庫頁面按鍵盤的 /　（斜線），
或點上方的搜尋框，選「In this repository」

② 本機：在編輯器裡按 Ctrl + Shift + F　全專案搜尋

要搜的四個字

`key`　`token`　`password`　`secret`

另外也搜一次你自己的：
有沒有任何個案姓名、身分證號、內部網址？

你應該看到什麼

四個關鍵字都搜不到任何「真的值」。

（搜到 .env.example 裡的
「你的金鑰放這裡」是正常的，
那不是真的金鑰。）

助教會實際搜一次——搜到即該項未通過。

沒看到的話，先檢查這幾件事

① 搜到了自己的金鑰
→ 第一件事：到服務商後台撤銷。
見第 20 頁的完整處理順序

② 搜到很多結果但看不懂
→ 逐一點開看是不是真的值。
變數名稱裡有 key 是正常的

這個動作要養成習慣：每次部署完就搜一次，三十秒。期末專題的評分規準裡，發現金鑰外流該項直接零分——不是扣分，是零分。因為那是不可逆的。

Step 5　資料分級演練：十題情境判斷

你現在要做什麼　▶　打開 資料 \W5_ 資料分級演練十題，逐題判斷並寫下理由

每一題要回答三件事

① 可以／不可以／要先問誰
② 理由是什麼（依據哪一條判準）
③ 如果不可以，替代做法是什麼

判斷時參考這三頁

第 24　頁　什麼算去識別化
第 25　頁　去識別化之後仍不該貼的三種情況
第 26　頁　資料分級決策樹

你應該看到什麼

十題都有判斷與理由。

理由比答案重要——
有些題目本來就沒有標準答案，
重點是你依據的是什麼判準。

第 ③ 項（替代做法）不得留白。

沒看到的話，先檢查這幾件事

① 有幾題想不出替代做法
→ 通用替代做法：用結構相同、內容虛構的假資料去問「做法」
② 答案跟參考解答不一樣
→ 先看理由。理由站得住腳就可以，這一題本來就有灰色地帶

第 ③ 項是這個演練真正的目的。「不可以」很容易說，難的是「那要怎麼辦」——而在單位裡，只會說不可以的人，通常會被繞過去。

Step 6　估算你這個專案的月成本

你現在要做什麼　▶　到你用的 AI 工具後台，看實際用量並填一張估算表

五個欄位

① 到目前為止（五週）實際用掉多少?
② 平均一週多少?　　＝ ① ÷ 5
③ 一個月大約多少?　＝ ② × 4
④ 乘上 1.5–2　倍（實際總是比估的多）
⑤ 我設的用量上限是多少?

到哪裡看用量

多數工具在「設定 → 用量 / Usage / Billing」看得到。
順便確認第 ⑤ 項：有沒有設上限或告警。
沒設的話，現在就設。

你應該看到什麼

一張填好的估算表，五個欄位。

重點不是數字精確，
而是你回單位被問到時
「說得出一個有根據的範圍」。

而且你已經設好了用量上限。

沒看到的話，先檢查這幾件事

① 工具後台看不到用量
　→ 免費方案可能不顯示。
　　那就估「對話次數」也可以
② 覺得這題沒意義
　→ 回單位第一個被問的三題之一。
　　答不出來，案子就推不動

第 ⑤ 項請務必完成。代理模式一次跑錯方向，可能在十分鐘內用掉你以為夠一個月的額度——設上限是唯一能真正防住的手段，而且只要一分鐘。

用量爆掉的三個典型情境（都可以避免）

情境	發生了什麼	怎麼避免
一次跑錯方向的長對話	「幫我重構整個專案」→ 它讀遍所有檔案、改了十幾個地方、你說不對、它再來一次…十分鐘用掉一週的量	① 事先 commit （VC4）② 限制範圍 ③ 要它先說明計畫再動手 ④ 設用量上限
對話越拖越長	同一個對話開了三小時，每一次回應都要把前面所有內容再讀一遍	換主題就開新對話，把需要的檔案重貼。 附帶好處：它比較不會忘記前面
描述不清楚導致的來回	「不是這樣」「還是不對」「再改一下」——每一輪都是完整的一次用量	VC1 的五行規格 + 驗收條件。 這是省用量最有效的方法，不是換便宜模型

三個情境的共同解法都是「前四週學過的東西」：先 commit、限制範圍、先說明再動手、把需求寫清楚。換句話說——把課上好，用量自然就省下來了，不必另外學省錢技巧。

卡住了？照症狀查 ／ 常見的四個誤解

症狀 / 誤解	怎麼處理
重新整理之後紀錄不見了	可能是無痕模式或 file:// 開檔。用部署後的網址測比較準； 並確認程式有沒有顯示「不會被保存」的提示
無痕模式下整頁掛掉	try/catch 沒包好。要求 AI 把「所有」 localStorage 存取都包起來
.env 還是出現在 git status	檢查 .gitignore 有沒有存檔；或它之前已被 commit 過（歷史裡還在）
誤解① 環境變數＝安全	前端專案建置時會把值直接打包進去，按 F12 照樣看得到。 環境變數只解決「不推上 GitHub」，沒解決「不送到瀏覽器」
誤解② 去識別化就可以貼	小樣本、罕見組合、單位規定——三種情況下仍然不可以。 替代做法：用結構相同、內容虛構的假資料問做法
誤解③ 金鑰外流刪 commit 就好	刪不掉。歷史與鏡像早就存下來了。 唯一有效的處置是到服務商後台撤銷並重新產生
誤解④ 用便宜的模型最省錢	省用量最有效的方法是「一次講清楚」。 描述不清楚導致的來回重試，才是隱形的大宗

前三個誤解有一個共同點：它們都讓人以為「已經處理好了」。環境變數設了、去識別化做了、 commit 刪了——三件事都做了，但風險一件都沒有真的解除。第四個則是反過來：以為換模型就能省錢，其實真正的成本在「講不清楚而反覆重試」。

本週自我檢核（答不出來請回去看標註的頁數）

Q 1 **資料存放的三個層級是什麼？判斷用哪一層的那一句話是什麼？**

瀏覽器本機／雲端資料庫／院內系統；「別人需不需要看到我存的東西」→ 第 5 頁與第 9 頁

Q 2 **為什麼靜態網站藏不住金鑰？環境變數為什麼不能解決這件事？**

送到瀏覽器的東西按 F12 都看得到；前端建置會把值打包進去 → 第 15 頁

Q 3 **金鑰不小心推上 GitHub 了，第一件事要做什麼？為什麼不是刪 commit？**

到服務商後台撤銷。刪 commit 跑不贏自動掃描機器人 → 第 20 頁

Q 4 **去識別化之後，哪三種情況仍然不該貼進 AI 工具？替代做法是什麼？**

小樣本／單位規定／「只是想讓 AI 看一下」；用假資料問做法 → 第 25 頁

這四題就是你回單位之後最可能被問的四題。建議實際講出來練一次——在會議上答得出來，跟腦中覺得「大概知道」，中間的落差很大。

本週交付與驗收條件

交付物	驗收條件（全部達成才算通過）	怎麼查
① **能存紀錄的成品**	紀錄功能可用、重新整理後仍在、上限 10 筆； 無痕模式下會顯示「不會被保存」的提示	當場操作四個情境； 對照 範例 \v1.1
② **無金鑰外流**	儲存庫全文搜尋 key 、 token 、 password 、 secret 都搜不到任何真實的值	助教實際搜一次。 搜到即該項未通過
③ **.env 設定**	.gitignore 有排除 .env ； .env.example 有推上去； （有部署平台的話）後台有設環境變數	看設定
④ **資料分級演練**	十題都有判斷與理由，且「替代做法」欄不得留白	文件檢查。理由比答案重要
⑤ **成本估算**	五個欄位填完，且已在工具後台設好用量上限	估算表 ＋ 後台截圖

> 第 ② 項是紅線：期末專題的評分規準裡，發現金鑰外流該項直接零分。現在這個專案不需要金鑰，所以應該很容易通過——但習慣要現在養成，等到真的有金鑰時才練就太晚了。

第 5 週重點回顧

1. 「存起來」不是一個功能，是一個治理決定
2. 資料三層：瀏覽器本機／雲端資料庫／院內系統。越高，要回答的問題越多
3. 靜態網站藏不住金鑰——送到瀏覽器的東西按 F12 都看得到
4. 金鑰外流的第一件事是「撤銷」，不是「刪 commit」
5. 去識別化不是萬能：小樣本、罕見組合、單位規定，三種情況仍不可貼
6. 省用量最有效的方法是「一次講清楚」——這也省你的時間

下一步　下一週：自選專題發表與交接。你會把六週的能力套到自己工作上的一個真實痛點，並寫出一份讓別人接得下去的交接文件。